

SECCIÓN 2: EL PATRÓN OBSERVER Y LA PROGRAMACIÓN REACTIVA

4. Introducción al Patrón Observer

➔ Librería de Project Reactor: Esta librería de Project Reactor (usa el patrón observer) **implementa** lo que es la especificación de Stream Reactivos de Java, para ser exactos.

➔ Hay dos librerías de que implementan la especificación de Stream reactivos, la que es **Rxjava** y **Project Reactor**. Sin embargo, Spring está basado en la segunda.

➔ Ejemplo: JPA no es implementado por Java. Java lo único que hace es que provee las interfaces necesarias para que JPA funcione. Sin embargo, son software o librerías de terceros como Hibernate o EclipseLink, las cuales hacen la implementación de estas interfaces y hacen que funcione. En el caso de Project Reactor es exactamente lo mismo.

➔ Conceptos básicos:

Patrón Observer:

Es un patrón de diseño de comportamiento

Permite definir una dependencia uno-a-muchos entre objetos

Cuando un objeto cambia su estado, todos sus dependientes son notificados automáticamente

Componentes principales:

- Observable/Subject: mantiene la lista de observers y los notifica
- Observer: interface que define cómo recibir actualizaciones
- ConcreteObserver: implementación específica que reacciona a las actualizaciones

Stream (Flujo de datos):

- Es una secuencia de elementos que pueden ser procesados
- Características principales:
 - Puede ser infinito o finito
 - Puede ser síncrono o asíncrono
 - Permite operaciones en pipeline (map, filter, reduce)
 - No almacena datos, los procesa en tiempo real

Publisher:

Fuente de datos que emite elementos

Controla cuándo y cómo se emiten los datos

Puede tener múltiples suscriptores

Responsabilidades:

Gestionar suscripciones

Emitir datos

Notificar errores

Señalar finalización

Subscriber(s):

Consume los datos emitidos por el Publisher

Puede procesar o transformar los datos recibidos

Operaciones típicas:

onNext(): recibe nuevo elemento

onError(): maneja errores

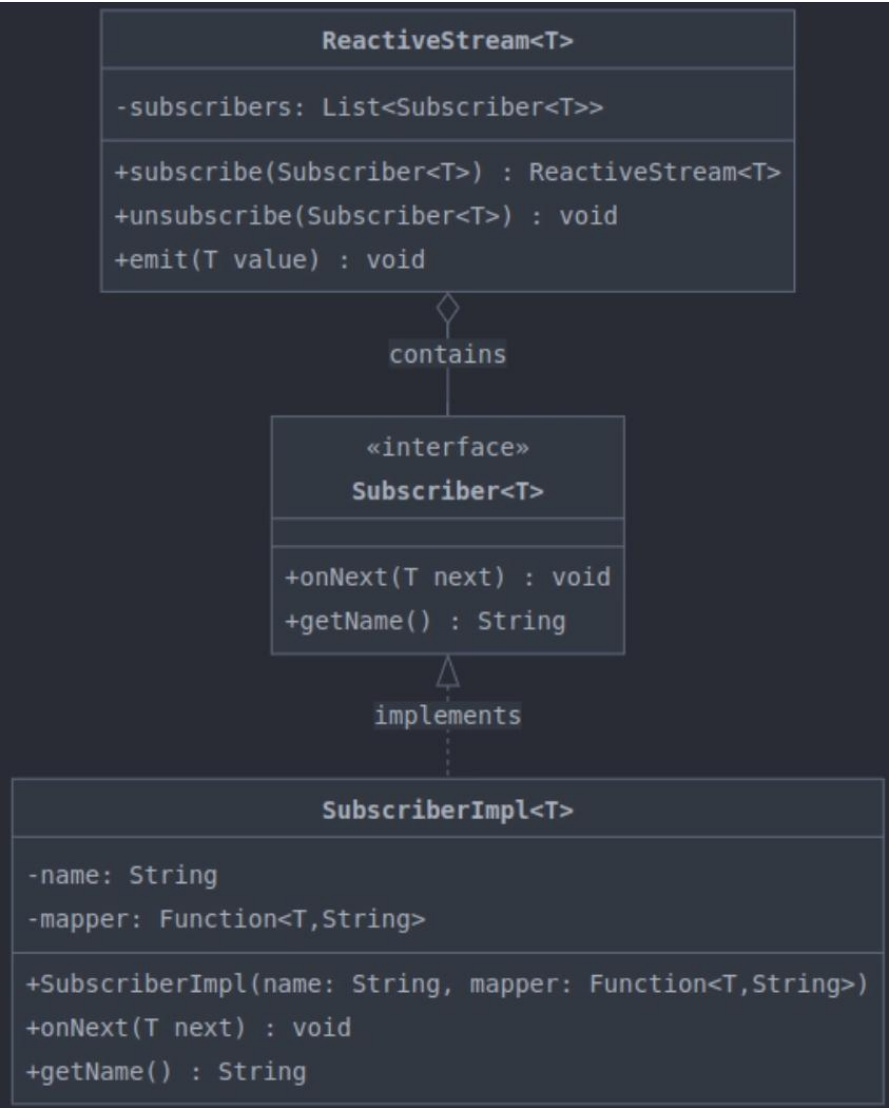
onComplete(): procesa finalización

Puede cancelar la suscripción cuando lo desees

Nota: El observable/subject es el ***publisher***. Los observers son los ***subscribers***

- El Observable/Subject
- Es el canal de YouTube
 - Puede crear y publicar nuevos videos (emit)
 - Mantiene una lista de suscriptores
 - Notifica a todos sus suscriptores

➔Diagrama de clase del patrón observer:



5. Iniciando el Proyecto

Name:

Location:

Project will be created in: D:\new_pro...subscriber-observer-demo

☐ Create Git repository

Build system: ☐ IntelliJ ☐ Maven ☐ Gradle

JDK: 21 Oracle OpenJDK 21.0.9

Gradle DSL: ☐ Kotlin ☐ Groovy

☐ Add sample code

Advanced Settings

Gradle distribution:

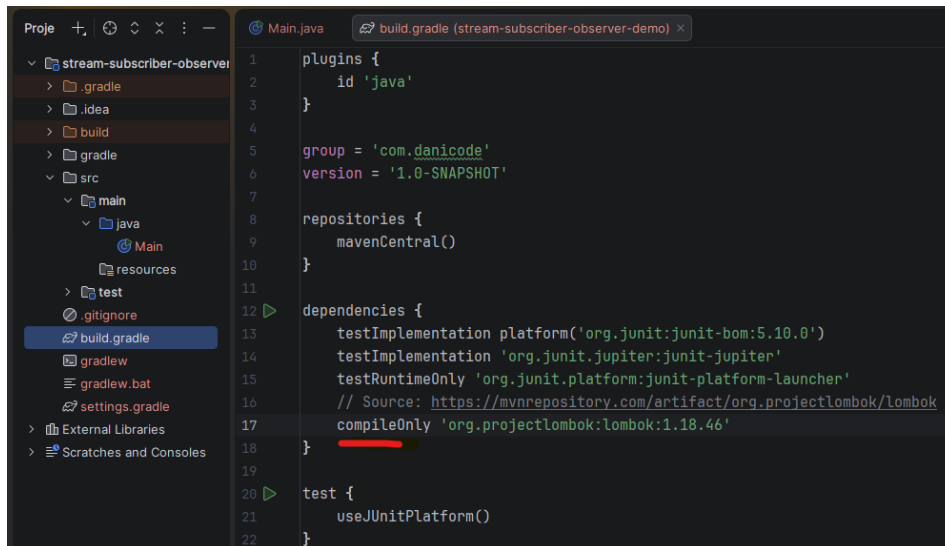
Gradle version: ☒ Auto-select

☒ Use these settings for future projects

GroupId:

ArtifactId:

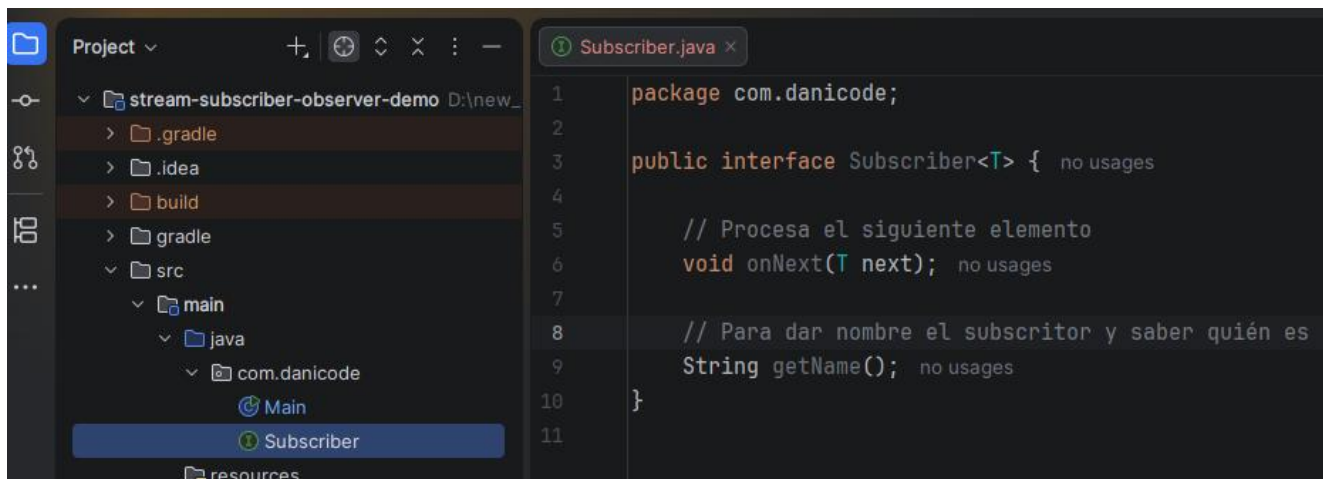
➔ Agregando paquete lombok



```
1 plugins {
2     id 'java'
3 }
4
5 group = 'com.danicode'
6 version = '1.0-SNAPSHOT'
7
8 repositories {
9     mavenCentral()
10 }
11
12 dependencies {
13     testImplementation platform('org.junit:junit-bom:5.10.0')
14     testImplementation 'org.junit.jupiter:junit-jupiter'
15     testRuntimeOnly 'org.junit.platform:junit-platform-launcher'
16     // Source: https://mvnrepository.com/artifact/org.projectlombok/lombok
17     compileOnly 'org.projectlombok:lombok:1.18.46'
18 }
19
20 test {
21     useJUnitPlatform()
22 }
```

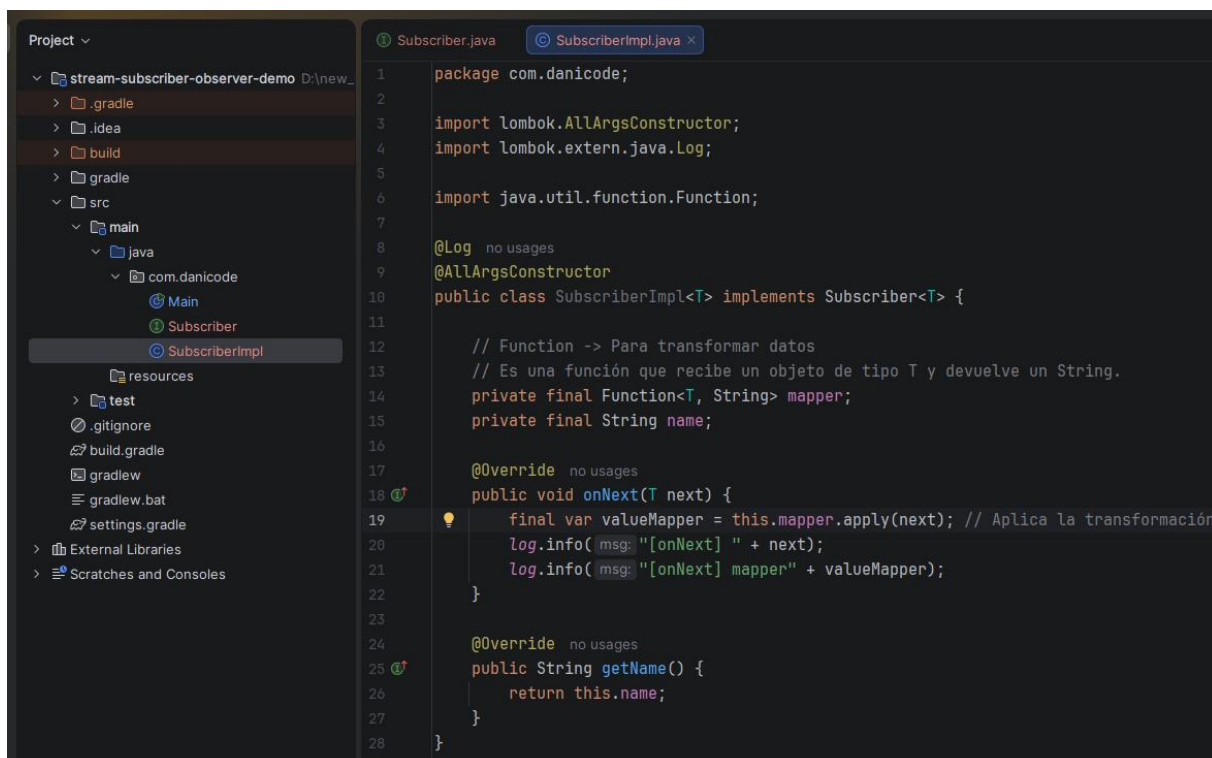
6. Implementando los Subscribers

➔ Creando la interfaz.



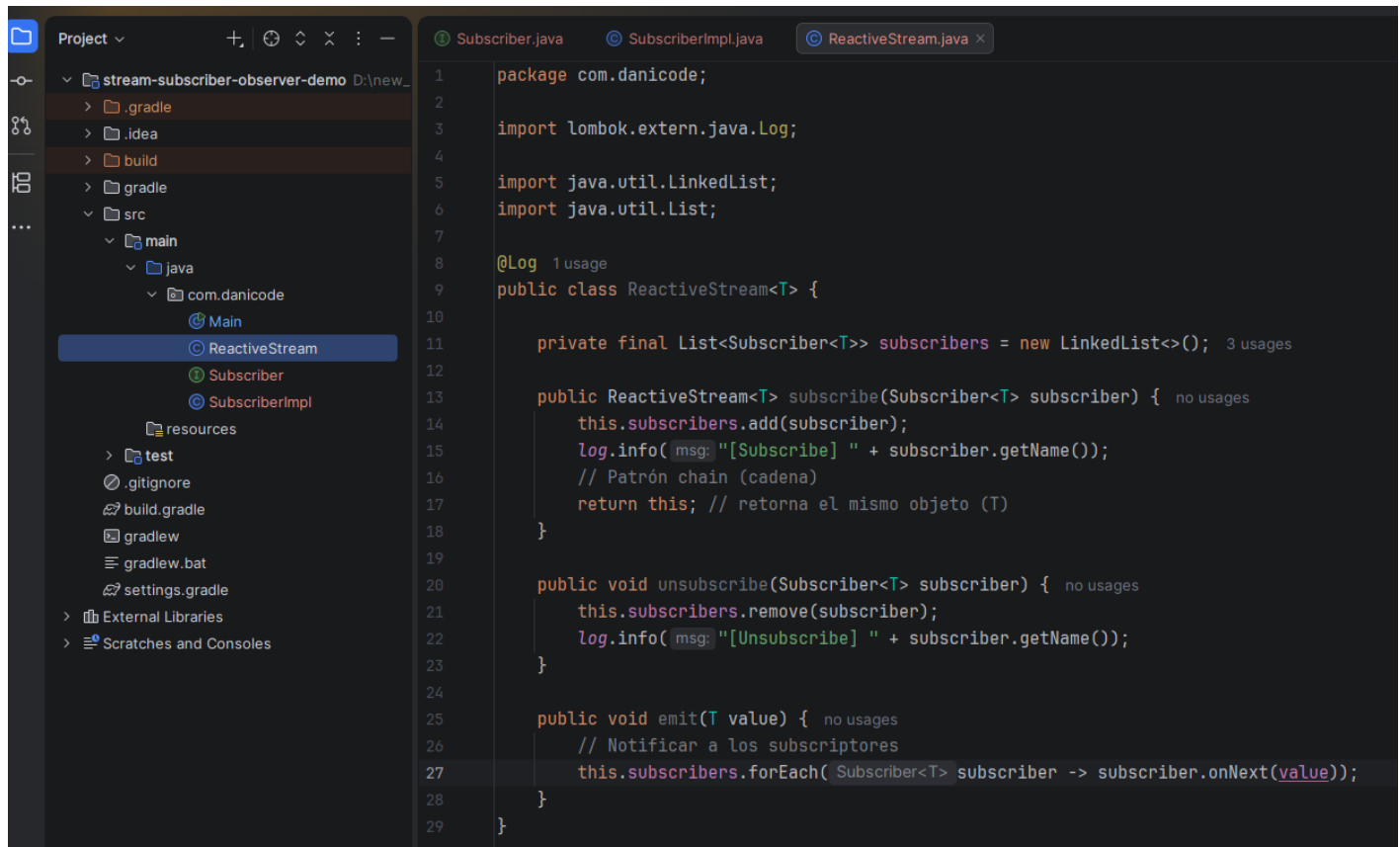
```
1 package com.danicode;
2
3 public interface Subscriber<T> {
4
5     // Procesa el siguiente elemento
6     void onNext(T next);
7
8     // Para dar nombre el suscriptor y saber quién es
9     String getName();
10 }
11
```

➔ Implementación:



```
1 package com.danicode;
2
3 import lombok.AllArgsConstructor;
4 import lombok.extern.java.Log;
5
6 import java.util.function.Function;
7
8 @Log
9 @AllArgsConstructor
10 public class SubscriberImpl<T> implements Subscriber<T> {
11
12     // Function -> Para transformar datos
13     // Es una función que recibe un objeto de tipo T y devuelve un String.
14     private final Function<T, String> mapper;
15     private final String name;
16
17     @Override
18     public void onNext(T next) {
19         final var valueMapper = this.mapper.apply(next); // Aplica la transformación
20         log.info(msg: "[onNext] " + next);
21         log.info(msg: "[onNext] mapper" + valueMapper);
22     }
23
24     @Override
25     public String getName() {
26         return this.name;
27     }
28 }
```

7. Implementando los Observers



```
1 package com.danicode;
2
3 import lombok.extern.java.Log;
4
5 import java.util.LinkedList;
6 import java.util.List;
7
8 @Log 1 usage
9 public class ReactiveStream<T> {
10
11     private final List<Subscriber<T>> subscribers = new LinkedList<>(); 3 usages
12
13     public ReactiveStream<T> subscribe(Subscriber<T> subscriber) { no usages
14         this.subscribers.add(subscriber);
15         log.info(msg: "[Subscribe] " + subscriber.getName());
16         // Patrón chain (cadena)
17         return this; // retorna el mismo objeto (T)
18     }
19
20     public void unsubscribe(Subscriber<T> subscriber) { no usages
21         this.subscribers.remove(subscriber);
22         log.info(msg: "[Unsubscribe] " + subscriber.getName());
23     }
24
25     public void emit(T value) { no usages
26         // Notificar a los subscriptores
27         this.subscribers.forEach( Subscriber<T> subscriber -> subscriber.onNext(value));
28     }
29 }
```

Nota: ReactiveStream ➔ Publisher

8. Creando el Publisher-Subscriber

➔ Creando 2 publisher, uno que emita datos enteros y otro de tipo Strings. Y se crearán 2 subscriptores para cada uno.

```
1 package com.danicode;
2
3 import lombok.extern.java.Log;
4
5 @Log &ydochoadev *
6 public class Main {
7     public static void main(String[] args) { &ydochoadev *
8         // Inmutables
9         final ReactiveStream<String> stringStream = new ReactiveStream<>(); // Publisher 1
10        final ReactiveStream<Integer> integerStream = new ReactiveStream<>(); // Publisher 2
11
12        final String subscriberName1 = "Subscriber 1";
13        final String subscriberName2 = "Subscriber 2";
14        final String subscriberName3 = "Subscriber 3";
15        final String subscriberName4 = "Subscriber 4";
16
17        // Primer subscriptor para el publisher de String
18        final Subscriber<String> stringSubscriber1 = new SubscriberImpl<>()
19            {
20                String str -> "Longitud: " + str.length(),
21                subscriberName1
22            };
23        // Segundo subscriptor para el publisher de String
24        final Subscriber<String> stringSubscriber2 = new SubscriberImpl<>()
25            {
26                String::toUpperCase,
27                subscriberName2
28            };
29        // Primer subscriptor para el publisher de Integer
30        final Subscriber<Integer> integerSubscriber1 = new SubscriberImpl<>()
31            {
32                Integer number -> "Value: " + number,
33                subscriberName3
34            };
35        // Segundo subscriptor para el publisher de Integer
36        final Subscriber<Integer> integerSubscriber2 = new SubscriberImpl<>()
37            {
38                Integer number -> "Potencia: " + (number * number),
39                subscriberName4
40            };
41    }
42 }
```

9. Implementando Suscripciones

➔Relacionar los publicadore con sus subscriptores

```
6 public class Main {
7     public static void main(String[] args) { &ydochoadev *
8         String str -> "Longitud: " + str.length(),
9         subscriberName1
10    };
11    // Segundo subscriptor para el publisher de String
12    final Subscriber<String> stringSubscriber2 = new SubscriberImpl<>()
13        {
14            String::toUpperCase,
15            subscriberName2
16        };
17    // Primer subscriptor para el publisher de Integer
18    final Subscriber<Integer> integerSubscriber1 = new SubscriberImpl<>()
19        {
20            Integer number -> "Value: " + number,
21            subscriberName3
22        };
23    // Segundo subscriptor para el publisher de Integer
24    final Subscriber<Integer> integerSubscriber2 = new SubscriberImpl<>()
25        {
26            Integer number -> "Potencia: " + (number * number),
27            subscriberName4
28        };
29
30    // Relacionando publisher y subscriber
31    stringStream
32        .subscribe(stringSubscriber1) // Método subscribe retorna el mismo objeto (patrón chain)
33        .subscribe(stringSubscriber2); // por ello se puede aplicar 2 veces el mismo método subscribe
34
35    integerStream
36        .subscribe(integerSubscriber1)
37        .subscribe(integerSubscriber2);
38
39    }
40 }
```

➔ Luego de colocar dicha relación, se debe emitir los datos, ya que el publicador aún no publica nada.

```
// Relacionando publisher y subscriber
stringstream
    .subscribe(stringSubscriber1) // Método subscribe retorna el mismo objeto (patrón chain)
    .subscribe(stringSubscriber2); // por ello se puede aplicar 2 veces el mismo método subscribe

integerStream
    .subscribe(integerSubscriber1)
    .subscribe(integerSubscriber2);

System.out.println("--- [Strings] ---");
// Emitiendo datos
stringstream.emit(value: "hello world");
stringstream.emit(value: "this is a string");
stringstream.emit(value: "teclado, mouse y monitor");

System.out.println("--- [Numbers] ---");
integerStream.emit(value: 5);
integerStream.emit(value: 10);
integerStream.emit(value: 15);
integerStream.emit(value: 12);

// Desuscribir
stringstream.unsubscribe(stringSubscriber1);
integerStream.unsubscribe(integerSubscriber2);
```

10. Depurando el Flujo Completo

➔ Para solventar el error al ejecutar (por gradle). Para que muestre el log de lombok

